Construtores de Classes

Orientação a Objetos – DCC025

Profª. Bárbara de Melo Quintela
barbara@ice.ufjf.b

Universidade Federal de Juiz de Fora – UFJF
Departamento de Ciência da Computação - DCC

2012.1

Construtores

```java
public class Pessoa {

    private String nome;

    public Pessoa(String nomeTeste){
        nome = nomeTeste;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

Construtores

```
public class Pessoa {

    private String nome;

    public Pessoa(String nomeTeste){
        nome = nomeTeste;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

Construtor da classe: permite inicializar os campos declarados

Construtores

Classe Pessoa

```
public class Pessoa {

    private String nome;

    public Pessoa(String nomeTeste){
        nome = nomeTeste;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

```
    public static void main(String args[]){

        Pessoa p1 = new Pessoa("Edmar Oliveira");
        System.out.println(p1.getNome());
    }
}
```

Método main em outra Classe

Exemplo em C++

```cpp
5   class Pessoa
6   {
7       private:
8           string nome;
9       public:
10          Pessoa(string);
11          ~Pessoa();
12          string getNome();
13          void setNome(string);
14  };
15  Pessoa::Pessoa(string nomeTeste){
16      this->nome = nomeTeste;
17      cout << "Construtor de Pessoa Chamado" << endl;
18  }
19
20  Pessoa::~Pessoa()
21  {
22      cout << "Destrutor de Pessoa Chamado" << endl;
23  }
24
25  void Pessoa::setNome(string nome){
26      this->nome = nome;
27  }
28
29  string Pessoa::getNome(){
30      return this->nome;
31  }
```

```cpp
1   #include <iostream>
2   #include <string>
3   #include <stdio.h>
4   #include "Pessoa.h"
5
6   using namespace std;
7
8   int main(){
9       Pessoa pessoa1("Fulano de Tal");
10      cout << pessoa1.getNome() << endl;
11      return 0;
12  }
```

Exemplo em C++

E:\Documentos\Temporario\OO\TestaPessoa.exe

```
Construtor de Pessoa Chamado
Fulano de Tal
Destrutor de Pessoa Chamado

Process returned 0 (0x0)   execution time : 0.094 s
Press any key to continue.
```

Construtores

- Construtores de classes
  - Toda classe Java possuiu um construtor padrão ou um construtor especificado pelo programador. Todo campo em Java possui um valor inicial padrão (um valor fornecido pelo Java quando o programador não especificar o valor inicial do campo). Ex:
    - Campos do tipo String: possui NULL como valor inicial padrão
    - Tipos inteiros possui “0” como valor padrão.

Construtores

(a)

```java
public class Pessoa {

    private String nome;

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

```java
public class Pessoa {

    private String nome;

    public Pessoa(String nomeTeste){
        nome = nomeTeste;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

(b)

Construtores

- Continuação
  - Não se exige que campos sejam explicitamente inicializados antes de serem utilizados em um programa. Isso quer dizer que classes podem ser instanciadas sem se especificar os valores iniciais dos objetos criados.
    - A classe Pessoa (mostrada anteriormente) pode ser instanciada sem definir valores para os campos Nome, Telefone e Endereço.

Construtores

- Construtores padrão
  - Cada classe deve ter pelo menos um construtor. Se não for fornecido um construtores na classe, o compilador cria um construtor padrão que não recebe nenhum argumento quando é invocado.
  - Construtores tem o mesmo nome da classe. Por padrão os únicos métodos que iniciam com letra maiúscula

Classe “Sem” Construtor

```java
public class Pessoa {

    private String nome;

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

```java
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Pessoa p1 = new Pessoa();
8          System.out.println(p1.getNome());
9      }
10 }
```

O que será impresso?

Construtores

- Construtores padrão
  - Se uma classe declara , pelo menos, um construtor, o compilador não criará um construtor padrão para ela. Se o programador desejar, ele pode especificar uma inicialização padrão de objetos de sua classe.
    - Vantagem: Todos os objetos são inicializados com os mesmos valores
      Qualquer objeto instanciado possuirá os mesmos valores iniciais
    - Como se faz isso: declarando um construtor sem argumentos

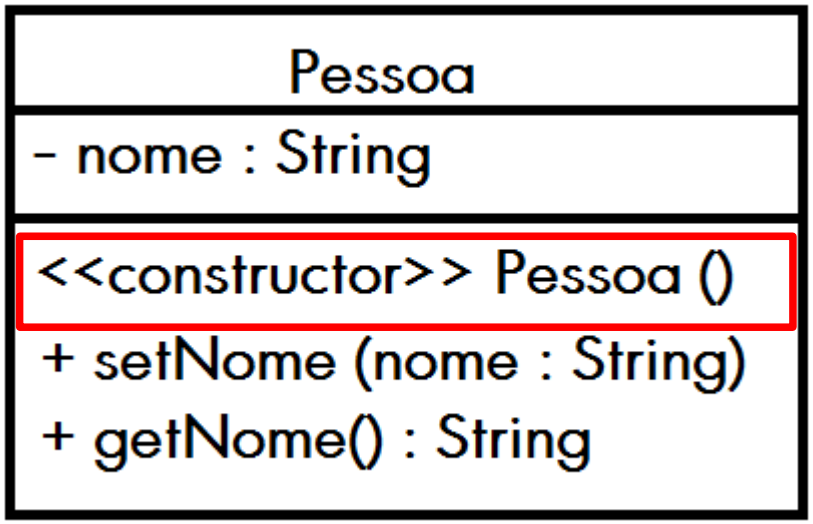
Classe Com Construtor

```
public class Pessoa {

    private String nome;

    public Pessoa(String nomeTeste){
        nome = nomeTeste;
    }

    public String getNome(){
        return nome;
    }

    public void setNome(String nome){
        this.nome = nome;
    }
```

A classe Pessoa possui um construtor. Logo, o compilador do Java não irá criar um construtor para ela

Construtor

Representação do contrutor no modelo:



Exemplos e Implicações

Construtores

```
public class Pessoa {

    private String nome;

    public Pessoa(String nome) {
        this.nome = nome;
    }
}
```

Classe Pessoa. Observe a presença de um construtor (com argumento)

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste");

    }
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa();

    }
}
```

The constructor Pessoa() is undefined

3 quick fixes available:

- Add argument to match 'Pessoa(String)'
- Change constructor 'Pessoa(String)': Remove parameter 'String'
- Create constructor 'Pessoa()'

Construtores

A cada objeto instanciado, um valor para o campo nome deve ser, obrigatoriamente, fornecido (caso contrário gera-se um erro)

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste1");
        Pessoa p2 = new Pessoa("NomeTeste2");
        Pessoa p3 = new Pessoa("NomeTeste3");
        Pessoa p4 = new Pessoa("NomeTeste4");

    }
}
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste1");
        Pessoa p2 = new Pessoa("NomeTeste2");
        Pessoa p3 = new Pessoa("NomeTeste3");
        Pessoa p4 = new Pessoa();

    }
}
```

The constructor Pessoa() is undefined

3 quick fixes available:

- Add argument to match 'Pessoa(String)'
- Change constructor 'Pessoa(String)': Remove parameter 'String'
- Create constructor 'Pessoa()'

Construtores

```
public class Pessoa {

    private String nome;

    public Pessoa() {
        this.nome = "Nome Teste";
    }
}
```

Observe a presença de um construtor, mas sem argumentos (parâmetros)

```
public class Pessoa {

    private String nome;

    public Pessoa(String nome) {
        this.nome = nome;
    }
}
```

Observe a presença de um construtor, mas com argumentos (parâmetros)

Prática

Construtor

1. Crie uma classe chamada **Data** com três variáveis de instância: **mês** (tipo int), **dia** (tipo int) e **ano** (tipo int).

2. Forneça um construtor que inicializa as três variáveis de instância.

3. Forneça um método set e um método get para cada variável de instância.

4. Forneça um método **mostraData** que mostra dia, mês e ano separados por barras normais (/).

5. Crie um aplicativo **TestaData** para demonstrar as capacidades da classe **Data**.

Resumo Construtores

Construtor Inexistente

1ª possibilidade – Sem Construtor

O compilador cria um construtor padrão para a classe
As variáveis de instância são inicializadas com seus valores padrão

```
public class Pessoa {

    private String nome;
}
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa();
    }
}
```

Construtor Sem Argumentos

2ª possibilidade – Construtor sem Argumentos

O programador pode definir um valor padrão para as variáveis

```
public class Pessoa {

    private String nome;

    public Pessoa() {
        this.nome = "Nome Teste";
    }
}
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa();
    }
}
```

Construtor Com Argumentos

3ª possibilidade – Com Construtor (Com Argumentos)

O compilador não cria um construtor padrão para a classe
As variáveis de instância são inicializadas os valores passados no construtor

```
public class Pessoa {

    private String nome;

    public Pessoa(String nome){
        this.nome = nome;
    }
}
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste1");
    }
}
```

Construtor Com Alguns Argumentos

4ª possibilidade – Com Construtor (Com Alguns Argumentos de Inicialização)

O compilador não cria um construtor padrão para a classe
As variáveis de instância são inicializadas os valores passados no construtor
As variáveis não inicializadas pelo construtor são inicializadas com seus valores padrão

```
public class Pessoa {

    private String nome;
    private String sobrenome;

    public Pessoa(String nome){
        this.nome = nome;
    }

    public String getNome() {
        return nome;
    }

    public String getSobrenome() {
        return sobrenome;
    }
}
```

```
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste1");
    }
}
```

Construtores Sobrecarregados

Construtores Sobrecarregados

```
public class Pessoa {

    private String nome;
    private String sobrenome;

    public Pessoa(String nome){
        this.nome = nome;
    }

    public Pessoa(String nome,String sobrenome){
        this.nome = nome;
        this.sobrenome = sobrenome;
    }
}
```

Sobrecarga de construtores

Sobrecarga de métodos

Métodos de nomes iguais, mas assinaturas diferentes
São métodos diferentes, que apenas posssuem o mesmo nome

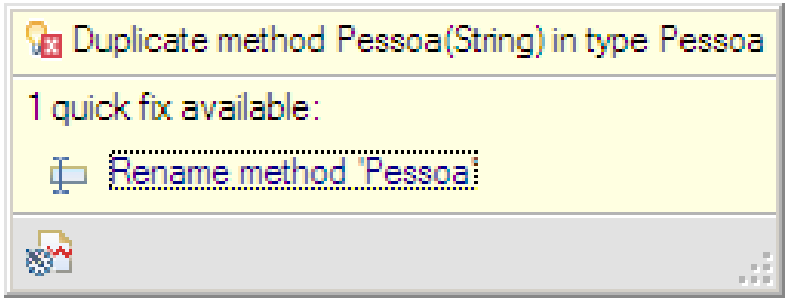
Construtores Sobrecarregados

```
class Pessoa {
    String rg;
    int cpf;

    Pessoa(String rg){
        this.rg = rg;
    }

    Pessoa(int cpf){
        this.cpf = cpf;
    }
}
```

```
// Chamando o primeiro construtor
Pessoa p1 = new Pessoa("123456X");
// Chamando o segundo construtor
Pessoa p2 = new Pessoa(123456789);
```

Construtores Sobrecarregados

```java
public class Pessoa {

    private String nome;
    private String sobrenome;

    public Pessoa(){
        this(null,null);
    }

    public Pessoa(String nome){
        this(nome,null);
    }

    public Pessoa(String sobrenome){
        this(null,sobrenome);
    }

    public Pessoa(String nome,String sobrenome){
        setNome(nome);
        setSobrenome(sobrenome);
    }
```



Mescla de Construtores

```java
public class Pessoa {

    private String nome;

    public Pessoa() {
        this("teste");
    }

    public Pessoa(String nome){
        setNome(nome);
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
}
```

1

2

3

4

```java
public class Executa {

    public static void main(String args[]){
        Pessoa p1 = new Pessoa("NomeTeste1");
        Pessoa p2 = new Pessoa();

        System.out.println(p1.getNome());
        System.out.println(p2.getNome());

    }
}
```

1 - Construtor sem argumentos
invoca construtor com argumento

2 - Construtor com argumentos
invoca método setNome

3 - Método getNome
4 - Método setNome

Construtor Chamando Construtor

```
1   // arquivo: Conta.java
2   class Conta {
3       int numero;
4       double limite;
5
6       Conta(int numero) {
7           this.numero = numero;
8       }
9
10      Conta(int numero, double limite) {
11          this(numero);
12          this.limite = limite;
13      }
14  }
```

Na linha 11, o this(numero) evita que o programador escreva novamente tudo que foi escrito no outro construtor. Neste caso, evita-se que seja escrito “this.numero = numero”.

Permite reusar construtores já escritos para inicializar atributos

Exercícios

Exercício 01

- Exercício 01
  - Escreva uma classe em Java que simule uma calculadora bem simples. Essa classe deve ter como atributos duas variáveis Double e uma String. Deve possuir um construtor que recebe como parâmetro dois números e um caracter, correspondente a uma das operações básicas (+, -, *, /). Deve ter um método para calcular a operação desejada e um para imprimir o resultado. O programa deve considerar divisões por zero como sendo erros, e imprimir uma mensagem adequada.
  - Excreva uma classe de Teste para testar seu programa.
    - Teste as quatro operações
    - Teste uma divisão por 0

Solução

```
3  public class Calculadora {
4
5      private double valor01;
6      private double valor02;
7      private String oper;
8
9      public Calculadora(int v1, int v2, String op){
10         this.valor01 = v1;
11         this.valor02 = v2;
12         this.oper = op;
13     }
14
15     public void calcularOperacao(){
16         double resultado = 0;
17
18         if(oper == "*"){
19             resultado = valor01 * valor02;
20         }
21         else if(oper == "+"){
22             resultado = valor01 + valor02;
23         }
24         else if(oper == "-"){
25             resultado = valor01 - valor02;
26         }
```

Solução - Continuação

```
27          else if(oper == "/"){
28              if (valor02 == 0){
29                  System.out.println("Divisão por 0 - Inválida");
30              }
31              else{
32                  resultado = valor01 / valor02;
33              }
34          }
35
36          imprimeresultado(resultado);
37      }
38
39      public void imprimeresultado(double resultado){
40          System.out.println("Resultado da operação: " + resultado);
41      }
42
43  }
```

Solução – Classe Principal

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Calculadora c1 = new Calculadora(100,10,"/");
8          c1.calcularOperacao();
9      }
10 }
```